

AWS Lambda

A Complete Visual Walkthrough — From Creation to Execution

Executive Summary

Seven screenshots documenting a complete AWS Lambda workflow — from the service homepage through function creation, code editing, test event setup, and execution — in the **Asia Pacific (Mumbai) ap-south-1** region, account **Aarush (106596066490)**.

Aspect	Details
Function Name	firstlambda
Runtime	Python 3.x (Python)
Region	Asia Pacific (Mumbai) — ap-south-1
Account	Aarush (106596066490)
Execution Status	Success — statusCode 200
Response Body	"Hello from Lambda!"
Test Event	myfirstlambda (Private, Synchronous)

Screenshot-by-Screenshot Description

Screenshot 1: AWS Lambda Landing Page

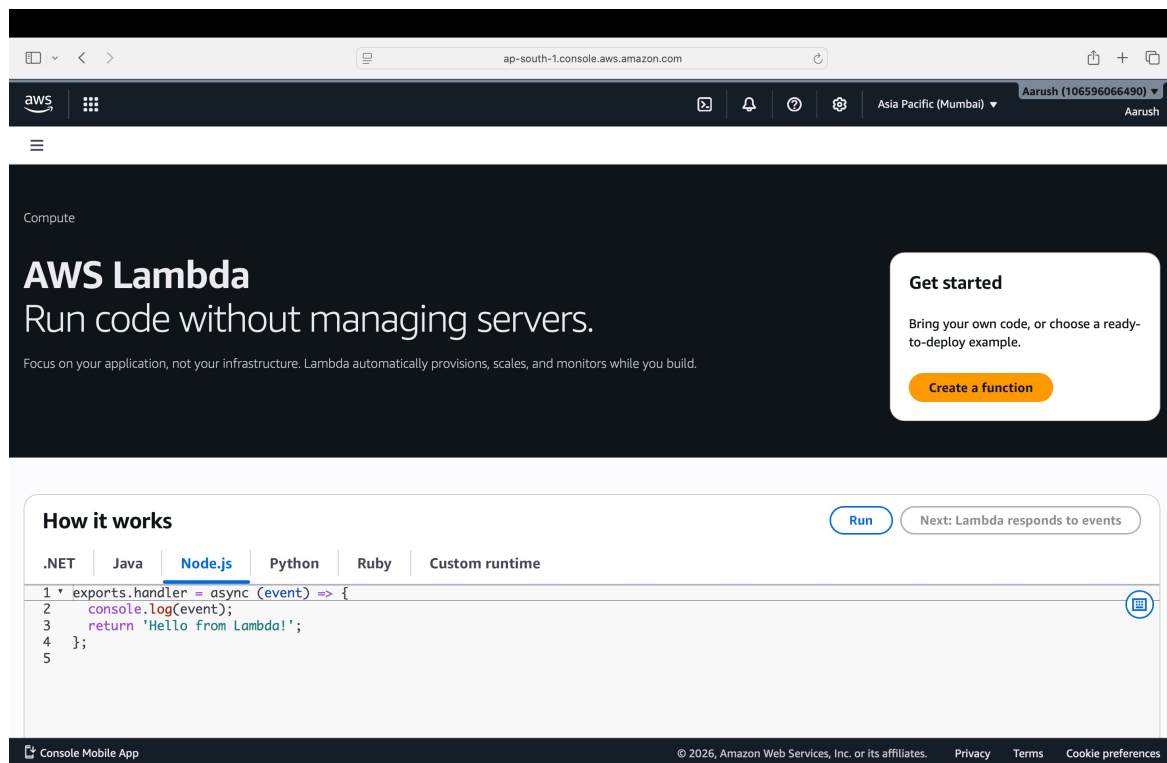


Figure: AWS Lambda Landing Page

AWS Lambda homepage with the "Run code without managing servers" banner and a "Create a function" button.

- Supports .NET, Java, Node.js, Python, Ruby, and Custom runtimes
- Serverless — no server management required
- Lambda auto-provisions, scales, and monitors execution

Screenshot 2: Create Function — Basic Configuration

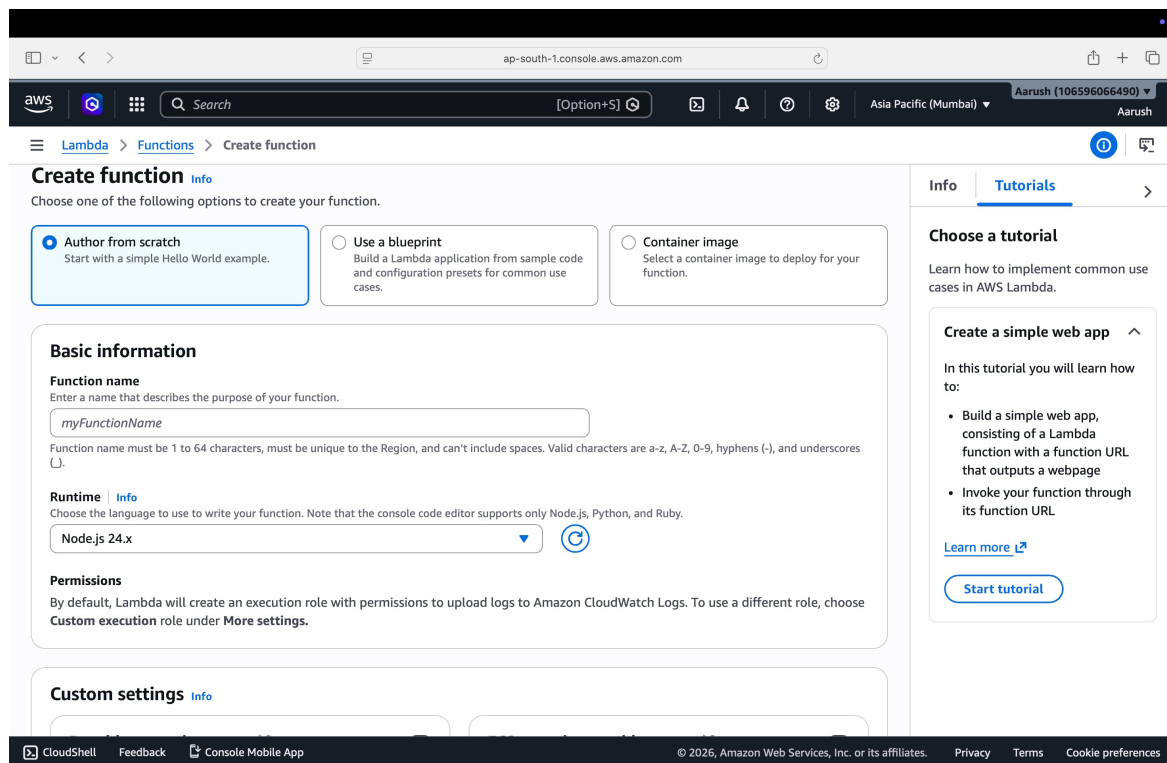


Figure: Create Function — Basic Configuration

The "Create function" page with "Author from scratch" selected, function name field, and runtime set to Node.js 24.x.

- Three creation options: Author from scratch, Use a blueprint, Container image
- Auto-created IAM execution role with CloudWatch Logs permission
- Function name: 1–64 chars, unique per region, no spaces

Screenshot 3: Runtime Selection Dropdown

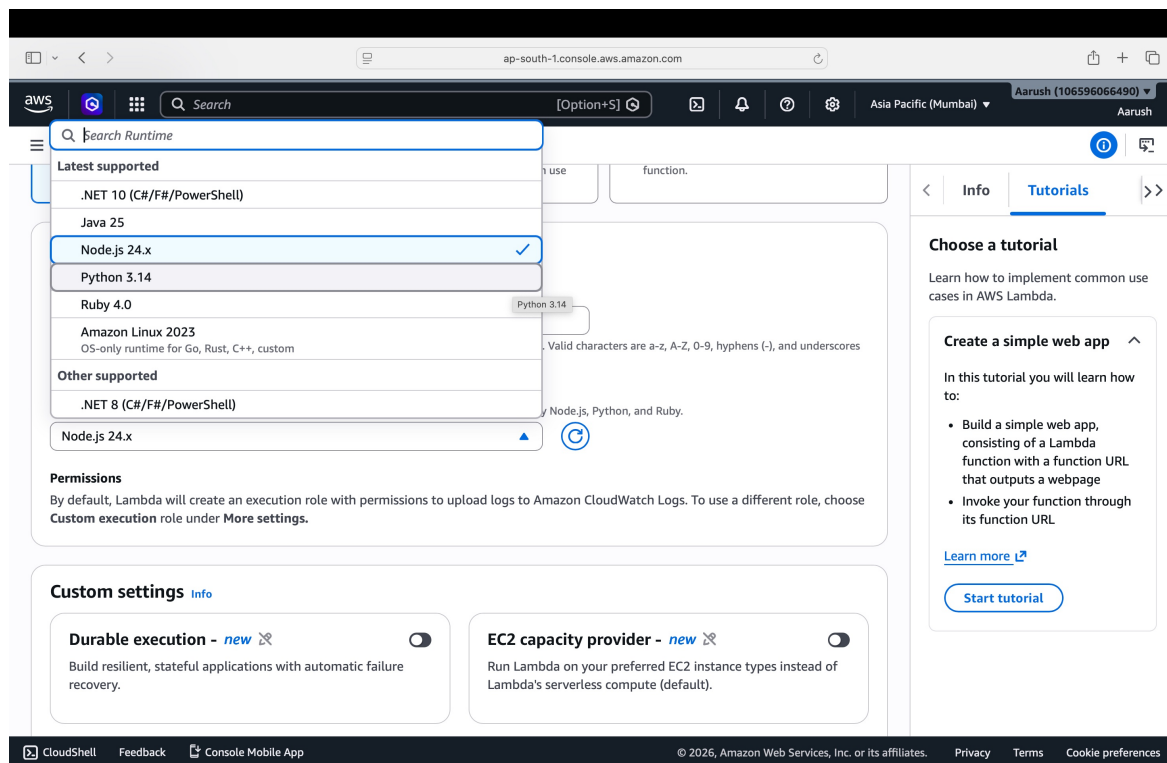


Figure: Runtime Selection Dropdown

Runtime dropdown open showing all supported runtimes grouped into "Latest supported" and "Other supported".

- Latest: .NET 10, Java 25, Node.js 24.x, Python 3.14, Ruby 4.0
- Amazon Linux 2023 for Go, Rust, C++, and custom runtimes
- Python 3.14 was chosen for this walkthrough

Screenshot 4: Function Created — Getting Started Dialog

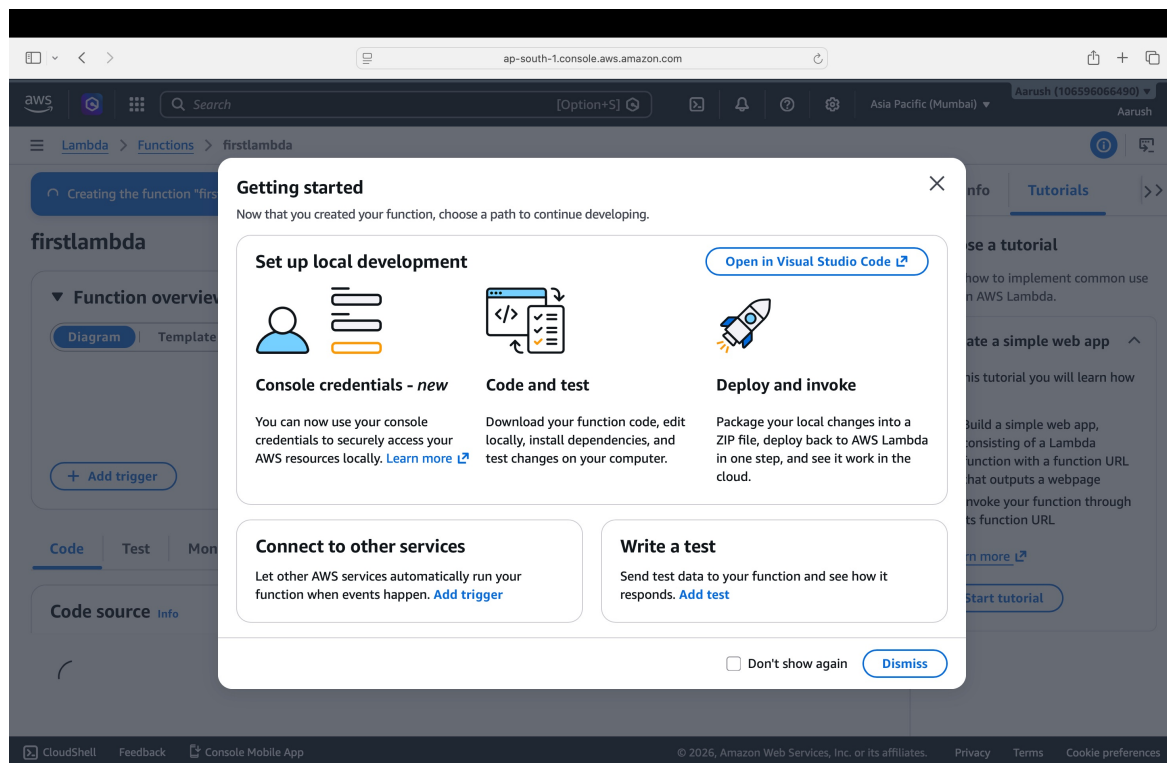


Figure: Function Created — Getting Started Dialog

"Getting started" modal shown after creating firstlambda, offering local development and cloud testing options.

- Console credentials: securely access AWS resources locally
- Code and test: edit locally, install dependencies, test
- Deploy and invoke: ZIP and redeploy changes to Lambda
- Add triggers or write test events directly from the console

Screenshot 5: Lambda Code Editor — Default Handler

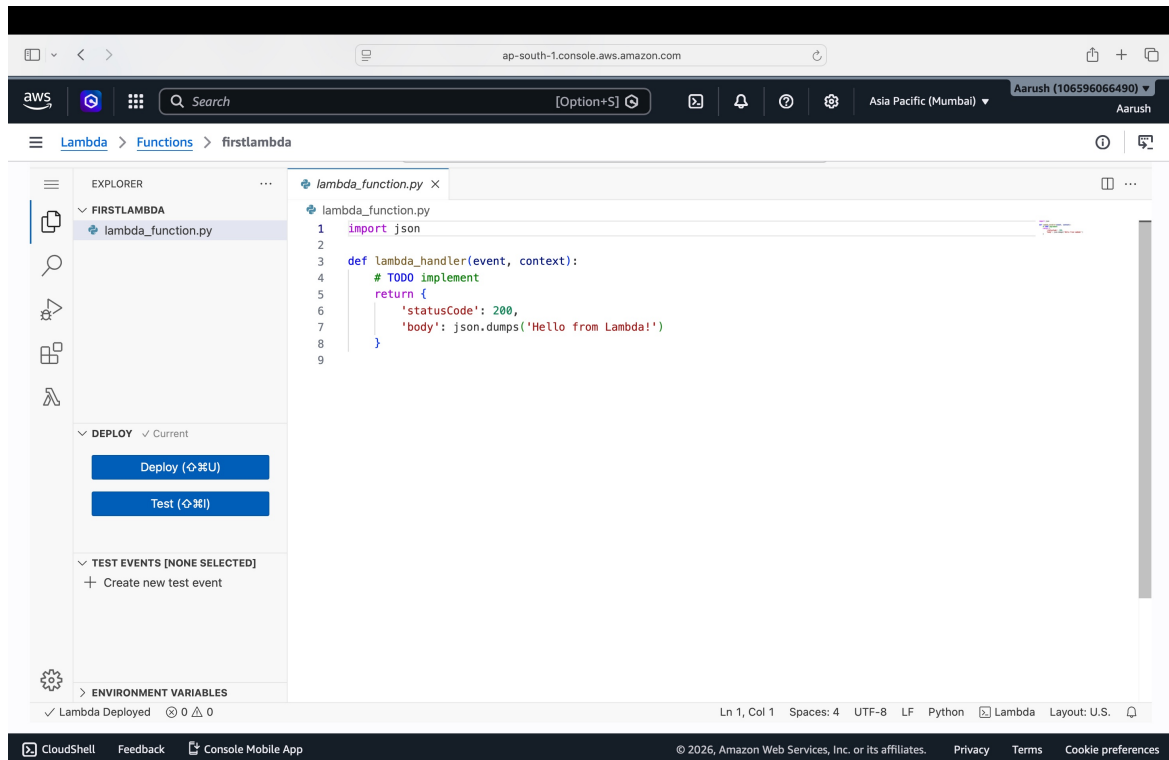


Figure: Lambda Code Editor — Default Handler

Inline VS Code editor showing the default lambda_function.py with a Hello World Python handler.

- Handler signature: lambda_handler(event, context)
- Returns statusCode 200 with body via json.dumps()
- Deploy and Test buttons available in the left panel

Screenshot 6: Create New Test Event Panel

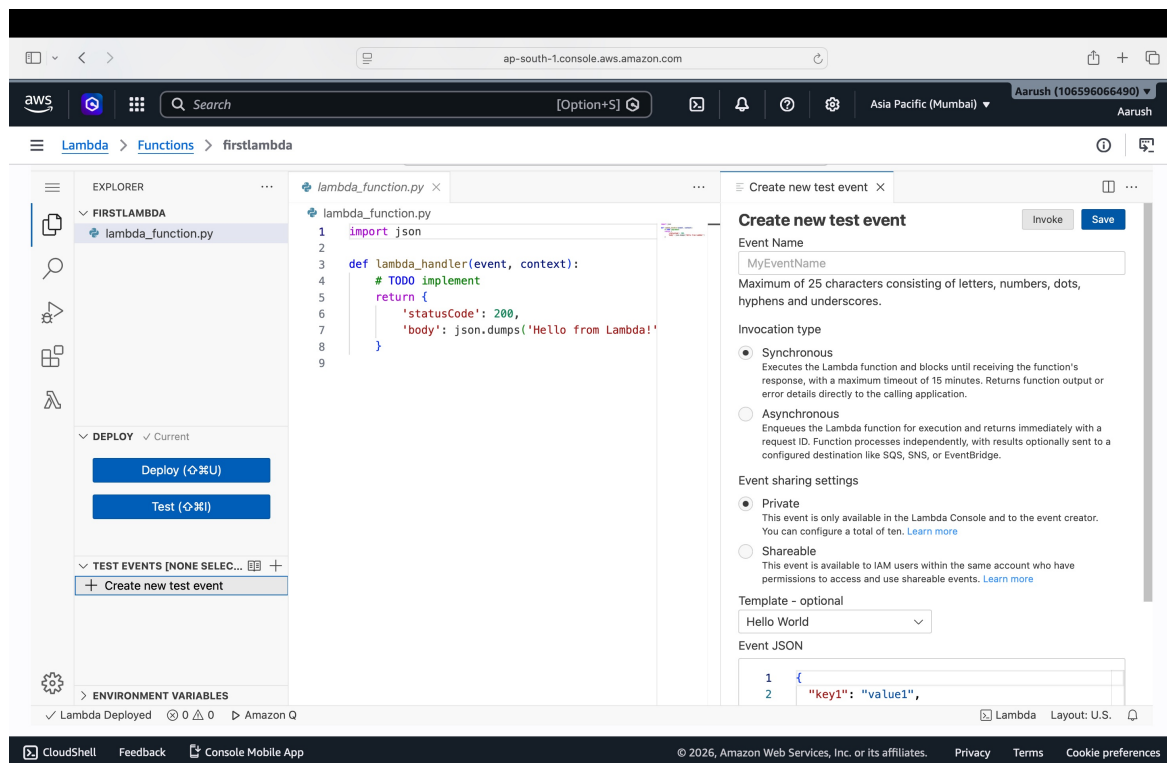


Figure: Create New Test Event Panel

Test event creation form with name, invocation type, sharing settings, template, and Event JSON fields.

- Synchronous: blocks until response; Asynchronous: returns request ID immediately
- Private (console only) or Shareable (IAM users in same account)
- Hello World template pre-loaded with sample JSON payload

Screenshot 7: Execution Result — Successful Response

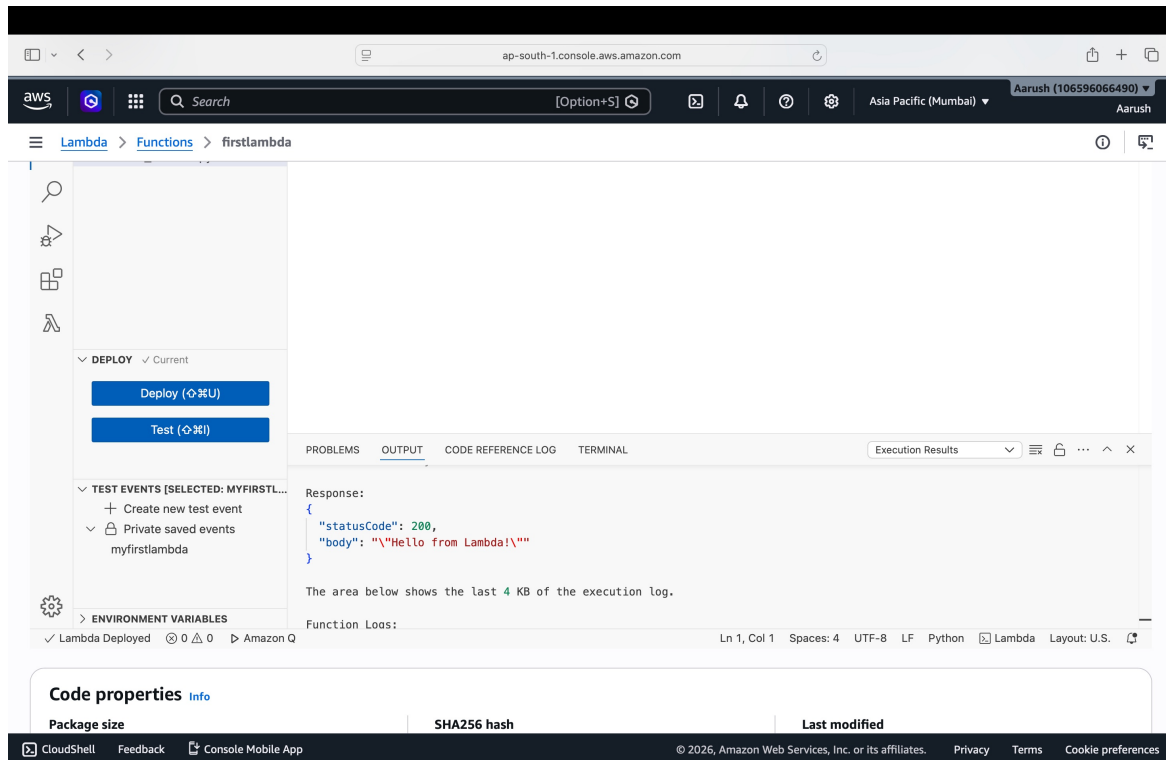


Figure: Execution Result — Successful Response

OUTPUT tab showing a successful execution with statusCode 200 and body "Hello from Lambda!".

- HTTP 200 — execution successful
- Function Logs show last 4 KB of CloudWatch output
- Status bar confirms: Lambda Deployed, 0 errors, 0 warnings

Key Concepts Illustrated

Serverless Computing

Write code; AWS handles all provisioning, scaling, and availability.

Event-Driven Execution

Functions are triggered by events (HTTP, S3, SQS, etc.) passed as the event object.

Handler Function

Entry point: `lambda_handler(event, context)` — event has trigger data, context has runtime info.

Synchronous vs Async

Sync waits for a response (max 15 min); Async queues the event and returns immediately.

IAM Execution Role

Default role grants CloudWatch Logs access; extra permissions must be added manually.

Test Events

Saved payloads (private or shareable) let you invoke the function repeatedly during development.

End-to-End Workflow Summary

- Step 1.** Navigate to AWS Lambda in the ap-south-1 (Mumbai) console
- Step 2.** Click "Create a function" on the Lambda homepage
- Step 3.** Choose "Author from scratch", name the function firstlambda, select Python 3.14 runtime
- Step 4.** Lambda auto-creates an IAM execution role with CloudWatch Logs permissions
- Step 5.** Dismiss the "Getting started" dialog after reviewing development options
- Step 6.** Open the inline VS Code editor; review the default lambda_function.py handler
- Step 7.** Click "Create new test event", name it myfirstlambda, use Hello World template, save
- Step 8.** Click "Test" (or Deploy then Test) to invoke the function synchronously
- Step 9.** Review the output: statusCode 200, body "Hello from Lambda!" — successful execution